Precision UI Hybrid Automation Framework

Team – TestOps

Project Source Code - GitHub

Table of Contents:

CONTENT	Page.NO
Project Overview & Objective	2
Tools & Technologies used	3
Framework Architecture (with diagram)	4
Test Scenarios & Test Cases	8
Execution steps and results	9
Challenges faced & solutions	12
Conclusion & future enhancements	13
Team Roles and Responsibilities	14

Project Overview & Objective

Project Overview

The **IPL Automation Framework** is a Selenium-based UI automation project developed to test and validate key functionalities of the IPL official website (https://www.iplt20.com/).

The framework automates critical modules such as:

- Footer links validation
- Team details verification
- Points table validation
- News search functionality

It is designed using the **Page Object Model (POM)** to ensure a modular, scalable, and maintainable structure.

Objective

- Automate real-time IPL website functionalities
- Reduce manual testing effort
- Ensure accuracy of UI elements and dynamic data
- Build a reusable and scalable automation framework
- Demonstrate industry-level automation practices

Tools & Technologies Used

Tools & Technologies Used	Purpose
Java (JDK 24)	Programming Language
Selenium WebDriver	Web UI Automation
TestNG	Test execution & assertions
Maven	Build & dependency management
WebDriverManager	Automatic driver setup
IntelliJ IDEA	Development IDE
GitHub	Version control

Framework Architecture

Design Pattern Used

Page Object Model (POM)

The framework is structured using POM to separate:

- Test logic
- Page interactions
- Utilities

This improves readability, reusability, and maintainability.

Architecture Diagram

Test Layer (TestNG)

↓

Page Object Model (POM)

↓

Base Class (Driver Setup)

↓

Utility Classes (DriverFactory, ScreenshotUtils, Listener)

↓

Selenium WebDriver

↓

Browser (Chrome / Edge)

↓

Reporting (Reports + Screenshots)

Architecture Explanation

1. Test Layer (TestNG)

- This is the top layer where all test cases are written.
- Uses TestNG framework for:

    - Test execution
    - Assertions
    - Test organization

In this project:

- FooterLinksTest
- NewsSearchTest
- PointsTableTest
- TeamDetailsTest

These classes define what to test

2. Page Object Model (POM)

- Contains page classes representing each web page
- Each class handles:
    - Locators (web elements)
    - Actions (click, search, etc.)

In this project:

- HomePage → Footer actions
- NewsPage → Search functionality
- StatsPage → Points table
- TeamsPage → Team details

This layer defines how to interact with the UI

3. Base Class (Driver Setup)

- Managed by BaseTest
- Handles:
    - Browser initialization
    - Opening application URL
    - Accepting cookies
    - Closing browser

Ensures a common setup for all test cases

4. Utility Classes

Reusable helper components:

DriverFactory

  - Manages WebDriver using ThreadLocal
  - Supports cross-browser execution

- ScreenshotUtils
  - Captures screenshots (full page and element level)
- TestListener
  - Listens for test failures
  - Automatically captures screenshots
- TeamData
  - Stores test data (team winning years)

Improves code reusability and maintainability

5. Selenium WebDriver

- Core automation engine
- Executes browser actions such as:
  - Click
  - SendKeys
  - Navigation

Acts as a bridge between test scripts and browser

6. Browser (Chrome / Edge)

- Actual execution happens in the browser
- Supported browsers:
  - Google Chrome
  - Microsoft Edge

Enables cross-browser testing

7. Reporting (Reports + Screenshots)

- Final output of test execution

Includes:

- TestNG reports (pass/fail status)
- Screenshots for failed tests

Stored in:

- target/surefire-reports/
- screenshots/

Helps in debugging and result analysis

Project Structure

```
IPL-Automation-Framework/
│── src/
│   ├── main/java/
│   │   ├── pages/
│   │   │   ├── HomePage.java
│   │   │   ├── NewsPage.java
│   │   │   ├── StatsPage.java
│   │   │   ├── TeamsPage.java
│   │   │
│   │   ├── utils/
│   │       ├── DriverFactory.java
│   │       ├── ScreenshotUtils.java
│   │       ├── TeamData.java
│
│   ├── test/java/
│   │   ├── base/
│   │   │   ├── BaseTest.java
│   │   │
│   │   ├── tests/
│   │   │   ├── FooterLinksTest.java
│   │   │   ├── NewsSearchTest.java
│   │   │   ├── PointsTableTest.java
│   │   │   ├── TeamDetailsTest.java
│   │   │
│   │   ├── utils/
│   │       ├── TestListener.java
│
│── testng.xml
│── testng-chrome.xml
│── testng-edge.xml
│── pom.xml
│── README.md
│── target/
│── screenshots/
```

Test Scenarios & Test Cases

1. Footer Links Validation

Test Case ID	Description	Steps
TC01	Verify footer sections exist	Scroll to footer
TC02	Validate links in each section	Retrieve links from each section

2. Team Details Verification

Test Case ID	Description	Steps
TC03	Verify team logos	Navigate to Teams page
TC04	Validate winning years	Hover over team

3. Points Table Validation

Test Case ID	Description	Steps
TC05	Validate top team name	Open points table page
TC06	Validate matches played	Retrieve matches value
TC07	Validate points logic	Compare matches and points

4. News Search Functionality

Test Case ID	Description	Steps
TC08	Search "Auction 2026"	Enter keyword and search

Execution Steps and Results

Execution Steps

1. **Clone the GitHub Repository**
   - Download or clone the project from GitHub
2. **Open Project in IDE**
   - Open the project in **IntelliJ IDEA**
3. **Install Maven Dependencies**
   - Ensure all dependencies are installed by running:

     mvn clean install
4. **Run Test Cases**

   Option 1: Using Maven

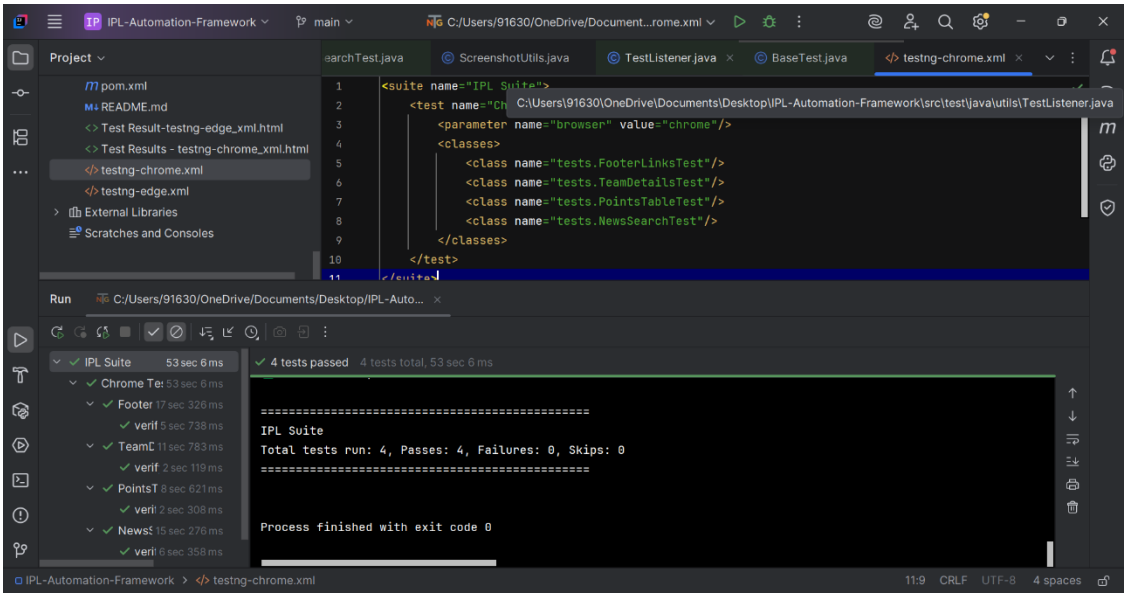
   mvn clean test

   Option 2: Using TestNG XML

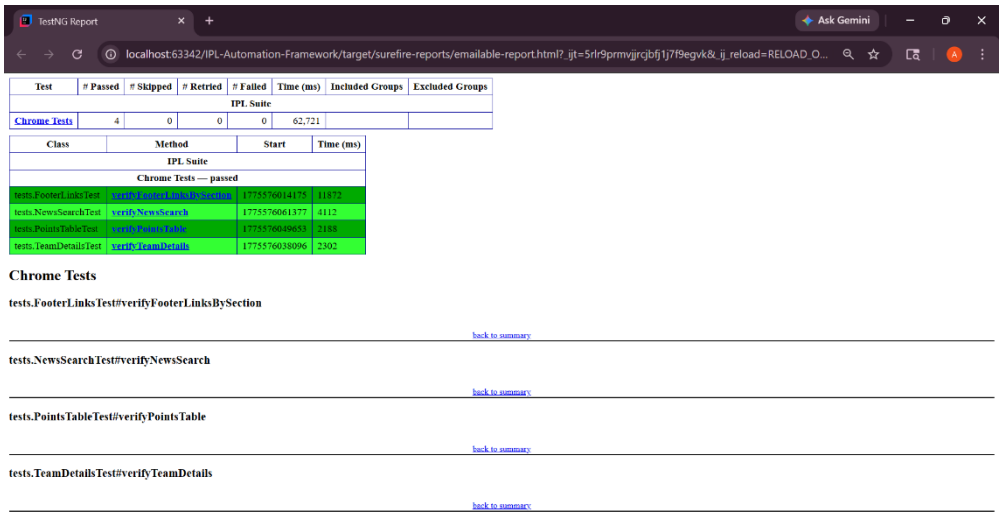
   mvn test -DsuiteXmlFile=testng-chrome.xml
5. **Execution Flow**
   - TestNG triggers test classes
   - BaseTest initializes WebDriver
   - Page classes perform UI actions
   - Utility classes handle screenshots and listeners
   - Results are generated

Execution Results

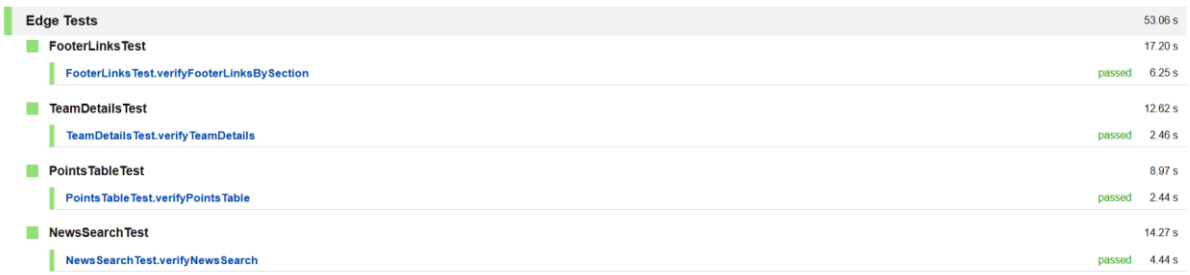
- All test cases executed successfully
- Console displays execution logs and status messages
- Screenshots are automatically captured for failed test cases
- TestNG reports are generated after execution



Result Report:



Report of Testing in Edge:



Report of testing in Chrome:

Chrome Tests		58.96 s
FooterLinksTest		17.52 s
FooterLinksTest.verifyFooterLinksBySection	passed	6.08 s
TeamDetailsTest		12.74 s
TeamDetailsTest.verifyTeamDetails	passed	3.01 s
PointsTableTest		13.08 s
PointsTableTest.verifyPointsTable	passed	3.06 s
NewsSearchTest		15.62 s
NewsSearchTest.verifyNewsSearch	passed	4.36 s

Challenges Faced & Solutions

During the development of the automation framework, several real-world challenges were encountered. The following table summarizes the issues and their solutions:

Challenge	Solution
Dynamic elements loading	Implemented WebDriverWait
Hidden elements (hover text)	Used JavaScriptExecutor
Screenshot capture on failure	Implemented TestNG Listener
Cross-browser execution	Used DriverFactory
Cookie popup handling	Created reusable method
Thread safety	Used ThreadLocal driver

Conclusion & Future Enhancements

Conclusion

The **IPL Automation Framework** successfully automates key functionalities of the IPL website such as footer links, team details, points table, and news search.

The framework is designed using the **Page Object Model (POM)**, which ensures:

- Better code structure
- Reusability
- Easy maintenance

It provides reliable test execution along with reporting and screenshot capture, making it effective for real-world automation testing.

Future Enhancements

- Add **Cucumber (BDD framework)** for behaviour-driven testing
- Integrate **Jenkins** for CI/CD automation
- Implement **parallel test execution** for faster testing
- Add **Allure reports** for better reporting visualization
- Extend framework to support **API testing (Rest Assured)**
- Enable execution using **Docker**

Team Roles and Responsibilities

Alekhya Kaki

- Designed overall framework structure
- Developed page classes and utility classes
- Implemented DriverFactory for browser setup
- Handled cross-browser execution (Chrome & Edge)
- Integrated TestNG for execution
- Worked on TestListener and screenshots
- Handled debugging and issue fixing
- Implemented Test cases 1 and 2 i.e. Footer Links Validation and Team Details Verification.

Roshitha Kurma

- Designed test scenarios and test cases
- Developed automation scripts using Selenium & TestNG
- Implemented Page Object Model (POM)
- Implemented Test cases 3 and 4 i.e. Points Table validation and News Search Functionality.
- Executed test cases and validated results
- Worked on reports and screenshots
- Created Readme file and Documentation of Project